

Modern Python

through



FastAPI

and friends

Who am I?

Sebastián Ramírez

tiangolo.com

Berlin, Germany



github.com/tiangolo



linkedin.com/in/tiangolo



twitter.com/tiangolo

Senior Staff Software Engineer at



External Consultant
for other teams

I created:



FastAPI



Typer



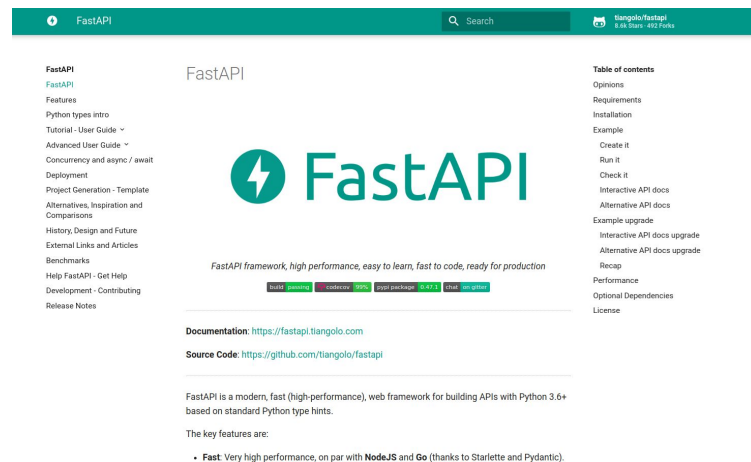
SQLModel



Asyncer



- Python Web API framework
- 50K+ GitHub stars (about 1K+ per month)
- Used by Microsoft, Uber, Netflix, etc.
- Performance in the top rank for Python
- 3rd most used (Python Developers Survey)
- Fastest growth (Python Developers Survey)
- 3rd most loved (StackOverflow Developer Survey)



Modern python™

Currently supported versions (3.7+)



f-strings



Type annotations



async / await



Performance



Community



FastAPI

- Pydantic
- HTTPX
- Starlette
- Uvicorn
- Uvloop



Typer

- Rich
- Click



SQLModel

- Pydantic
- SQLAlchemy



Modern Python



f-strings (formatted strings)





f-strings (formatted strings)

```
recipes = {  
    "crunchy-frog": {  
        "ingredients": [  
            "frogs",  
            "dew",  
            "spring water",  
            "cream milk chocolate",  
            "glucose",  
        ]  
    },  
    "albatross": {  
        "ingredients": [  
            "albatross"  
        ]  
    },  
}  
default_price = 1
```

```
def get_recipe(name, quantity=1):  
    recipe = recipes[name]  
    return {  
        "message": "Recipe for {name}".format(name=name),  
        "ingredients": recipe["ingredients"],  
        "total": default_price * quantity,  
    }
```





f-strings (formatted strings)

```
get_recipe("crunchy-frog")
```

```
{'message': 'Recipe for crunchy-frog',  
 'ingredients': ['frogs',  
                 'dew',  
                 'spring water',  
                 'cream milk chocolate',  
                 'glucose'],  
 'total': 1}
```

```
get_recipe("crunchy-frog", quantity=2)
```

```
{'message': 'Recipe for crunchy-frog',  
 'ingredients': ['frogs',  
                 'dew',  
                 'spring water',  
                 'cream milk chocolate',  
                 'glucose'],  
 'total': 2}
```





f-strings (formatted strings)

```
def get_recipe(name, quantity=1):  
    recipe = recipes[name]  
    return {  
        "message": "Recipe for {name}".format(name=name),  
        "ingredients": recipe["ingredients"],  
        "total": default_price * quantity,  
    }
```

```
def get_recipe(name, quantity=1):  
    recipe = recipes[name]  
    return {  
        "message": f"Recipe for {name}",  
        "ingredients": recipe["ingredients"],  
        "total": default_price * quantity,  
    }
```





f-strings (formatted strings)

```
get_recipe("crunchy-frog")
```

```
{'message': 'Recipe for crunchy-frog',  
 'ingredients': ['frogs',  
                 'dew',  
                 'spring water',  
                 'cream milk chocolate',  
                 'glucose'],  
 'total': 1}
```

```
get_recipe("crunchy-frog", quantity=2)
```

```
{'message': 'Recipe for crunchy-frog',  
 'ingredients': ['frogs',  
                 'dew',  
                 'spring water',  
                 'cream milk chocolate',  
                 'glucose'],  
 'total': 2}
```





f-strings (formatted strings)



`"{stuff}".format(stuff=stuff)`



`f"{stuff}"`



Modern Python



Type annotations (type hints)





Type annotations (type hints)

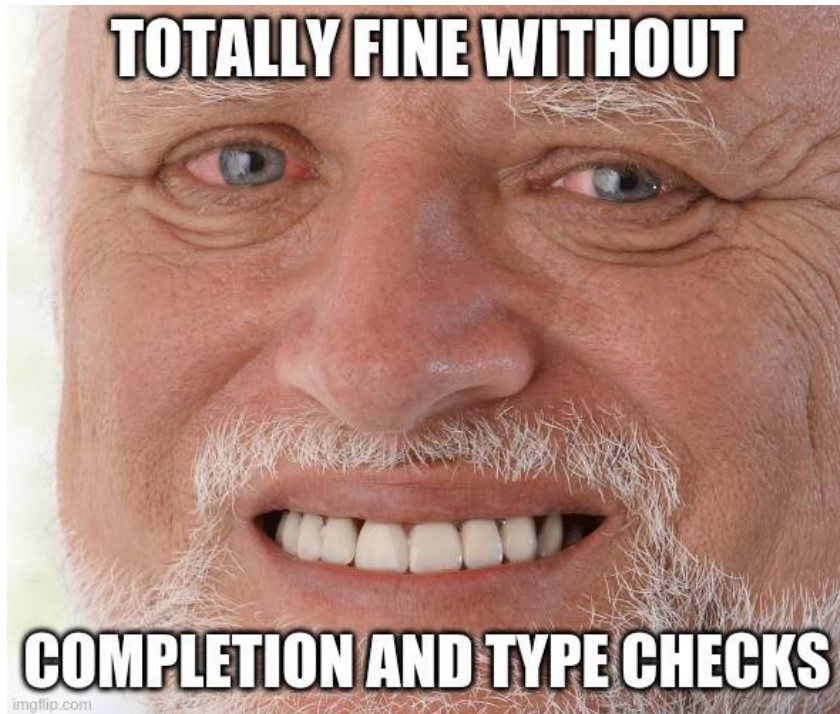
```
recipes = {  
    "crunchy-frog": {  
        "ingredients": [  
            "frogs",  
            "dew",  
            "spring water",  
            "cream milk chocolate",  
            "glucose",  
        ]  
    },  
    "albatross": {  
        "ingredients": [  
            "albatross"  
        ]  
    },  
}  
default_price = 1
```

```
def get_recipe(name, quantity=1):  
    name.  
    recip abc albatross  
    retur abc chocolate  
    " abc cream  
    " abc crunchy  
    " abc def  
    abc default_price  
    abc dew
```





Type annotations (type hints)





Type annotations (type hints)

```
def get_recipe(name, quantity=1):  
    recipe = recipes[name]  
    return {  
        "message": f"Recipe for {name}",  
        "ingredients": recipe["ingredients"],  
        "total": default_price * quantity,  
    }
```

```
def get_recipe(name: str, quantity=1):  
    recipe = recipes[name]  
    return {  
        "message": f"Recipe for {name}",  
        "ingredients": recipe["ingredients"],  
        "total": default_price * quantity,  
    }
```





Type annotations (type hints) - autocompletion

```
def get_recipe(name: str, quantity=1):  
    name.  
    recip  replace  
    retur  capitalize  
    "  casefold  
    "  center  
    "  count  
    }  encode  
     endswith
```





Type annotations (type hints) - autocompletion

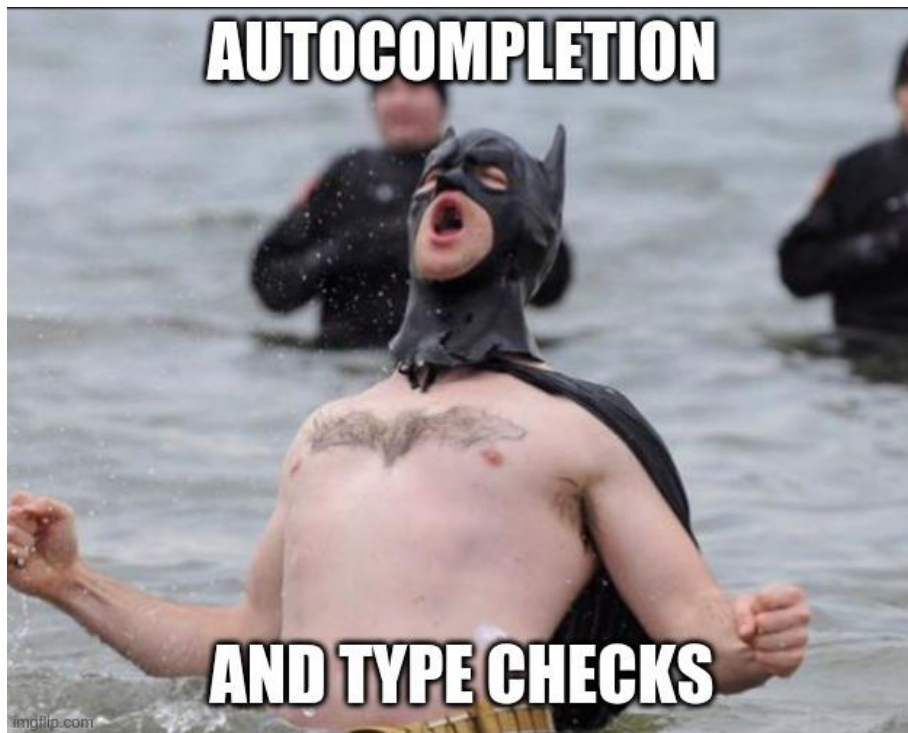
```
def get_recipe(name: str, quantity=1):  
    recipe = recipes[name]  
    title = name.replace("-", " ").  
    return {  
        "message": f"Recipe for {ti  
        "ingredients": recipe["ingr  
        "total": default_price * qu  
    }
```

- title
- replace
- capitalize
- casefold
- center
- count





Type annotations (type hints) - autocompletion





Type annotations (type hints)

```
def get_recipe(name: str, quantity=1):  
    recipe = recipes[name]  
    title = name.replace("-", " ").title()  
    return {  
        "message": f"Recipe for {title}",  
        "ingredients": recipe["ingredients"],  
        "total": default_price * quantity,  
    }
```





Type annotations (type hints)

```
get_recipe("crunchy-frog")
```

```
{'message': 'Recipe for Crunchy Frog',  
 'ingredients': ['frogs',  
                 'dew',  
                 'spring water',  
                 'cream milk chocolate',  
                 'glucose'],  
 'total': 1}
```





Type annotations (type hints)

```
def get_recipe(name: str, quantity=1):  
    recipe = recipes[name]  
    title = name.replace("-", " ").title()  
    return {  
        "message": f"Recipe for {title}",  
        "ingredients": recipe["ingredients"],  
        "total": default_price * quantity,  
    }
```

```
def get_recipe(name: str, quantity=None):  
    recipe = recipes[name]  
    title = name.replace("-", " ").title()  
    return {  
        "message": f"Recipe for {title}",  
        "ingredients": recipe["ingredients"],  
        "total": default_price * quantity,  
    }
```





Type annotations (type hints)

```
get_recipe("crunchy-frog")
```

TypeError

Traceback (most recent call last)

~/code/app/main.py in

----> 32 get_recipe("crunchy-frog")

~/code/app/main.py in get_recipe(name, quantity)

```
21         "message": f"Recipe for {title}",  
22         "ingredients": recipe["ingredients"],  
----> 23         "total": default_price * quantity,  
      24     }
```

TypeError: unsupported operand type(s) for *: 'int' and 'NoneType'





Type annotations (type hints) - error detection

```
def get_recipe(name: str, quantity=None):  
    recipe = recipes[name]  
    title = name.replace("-", " ").title()  
    return {  
        "message": f"Recipe for {title}",  
        "ingredients": recipe["ingredients"],  
        "total": default_price * quantity,  
    }
```

```
def get_recipe(name: str, quantity: int = None):  
    recipe = recipes[name]  
    title = name.replace("-", " ").title()  
    return {  
        "message": f"Recipe for {title}",  
        "ingredients": recipe["ingredients"],  
        "total": default_price * quantity,  
    }
```





Type annotations (type hints) - error detection

```
def get_recipe(name: str, quantity: int = None):
```

```
    recipe = recipe(title = name, default_price: Literal[1]
```

```
    return {
```

```
        "message": Right operand is of type "Optional[int]" mypy(note)
```

```
        "ingredient": Peek Problem (Alt+F8) No quick fixes available
```

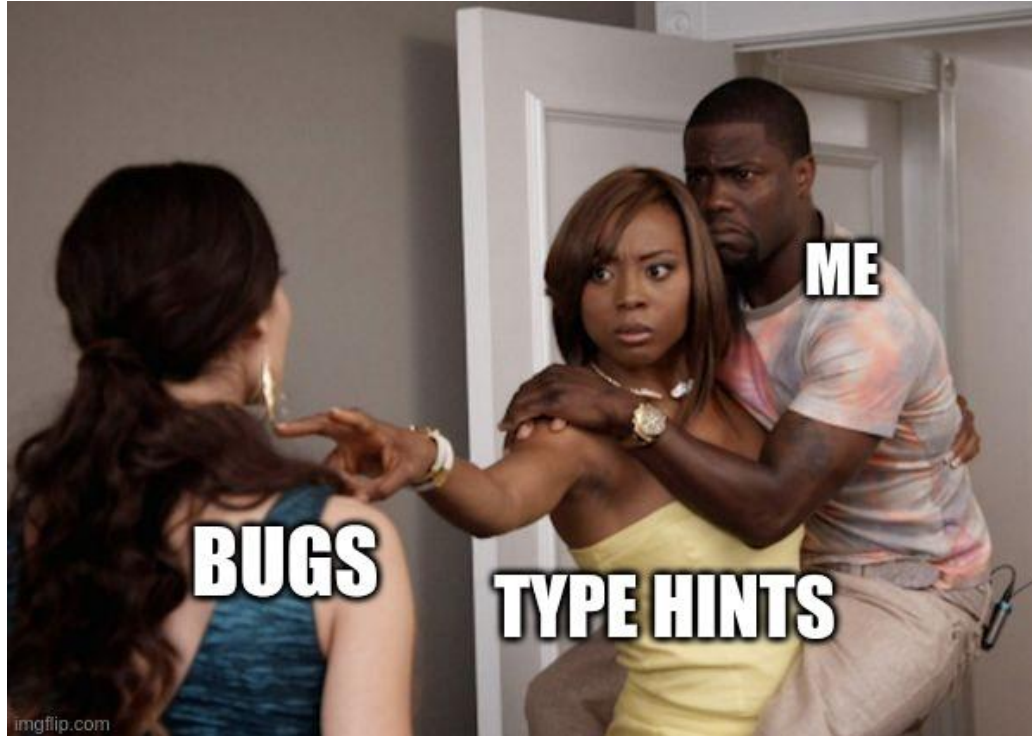
```
        "total": default_price * quantity,
```

```
    }
```





Type annotations (type hints)





Type annotations (type hints) - error detection

```
def get_recipe(name: str, quantity: int = None):  
    recipe = recipes[name]  
    title = name.replace("-", " ").title()  
    return {  
        "message": f"Recipe for {title}",  
        "ingredients": recipe["ingredients"],  
        "total": default_price * quantity,  
    }
```

```
def get_recipe(name: str, quantity: int = None):  
    recipe = recipes[name]  
    title = name.replace("-", " ").title()  
    result = {  
        "message": f"Recipe for {title}",  
        "ingredients": recipe["ingredients"],  
    }  
    if quantity is not None:  
        result["total"] = quantity * default_price  
    return result
```





Type annotations (type hints) - error detection

```
def get_recipe(name: str, quantity: int = None):  
    recipe = recipes[name]  
    title = name.replace("-", " ").title()  
    return {  
        "message": f"Recipe for {title}",  
        "ingredients": recipe["ingredients"],  
        "total": default_price * quantity,  
    }
```

```
def get_recipe(name: str, quantity: int = None):  
    recipe = recipes[name]  
    title = name.replace("-", " ").title()  
    result = {  
        "message": f"Recipe for {title}",  
        "ingredients": recipe["ingredients"],  
    }  
    if quantity is not None:  
        result["total"] = quantity * default_price  
    return result
```



Type annotations (type hints) - error detection

```
def get_recipe(name: str, quantity: int = None):
```

```
    recipe = recipes[name]
```

```
    title = name.replace("-", " ").title()
```

```
    result = {
```

```
        "message": f"Recipe {title}"
```

```
        "ingredients": recipe["ingredients"]
```

```
    }
```

```
    if quantity is not None:
```

```
        result["total"] = quantity * default_price
```

```
    return result
```

(parameter) quantity: int

Incompatible types in assignment (expression has type "int", target has type "Sequence[str]") mypy(error)

Peek Problem (Alt+F8) No quick fixes available





Type annotations (type hints) - error detection

```
def get_recipe(name: str, quantity: int = None):  
    recipe = recipes[name]  
    title = name.replace("-", " ").title()  
    result = {  
        "message": f"Recipe for {title}",  
        "ingredients": recipe["ingredients"],  
    }  
    if quantity is not None:  
        result["total"] = quantity * default_price  
    return result
```

```
def get_recipe(name: str, quantity: int = None):  
    recipe = recipes[name]  
    title = name.replace("-", " ").title()  
    result: dict = {  
        "message": f"Recipe for {title}",  
        "ingredients": recipe["ingredients"],  
    }  
    if quantity is not None:  
        result["total"] = quantity * default_price  
    return result
```





Type annotations (type hints)

(parameter) quantity: `int` | `None`

```
def get_recipe(name: str, quantity: int = None):  
    recipe = recipes[name]  
    title = name.replace("-", " ").title()  
    result: dict = {  
        "message": f"Recipe for {title}",  
        "ingredients": recipe["ingredients"],  
    }  
    if quantity is not None:  
        result["total"] = quantity * default_price  
    return result
```





Type annotations (type hints)

```
def get_recipe(name: str, quantity: int = None):  
    recipe = recipes[name]  
    title = name.replace("-", " ").title()  
    result: dict = {  
        "message": f"Recipe for {title}",  
        "ingredients": recipe["ingredients"],  
    }  
    if quantity is not None (parameter) quantity: int  
        result["total"] = quantity * default_price  
    return result
```





Type annotations (type hints)

```
def get_recipe(name: str, quantity: int = None):  
    recipe = recipes[name]  
    title = name.replace("-", " ").title()  
    result: dict = {  
        "message": f"Recipe for {title}",  
        "ingredients": recipe["ingredients"],  
    }  
    if quantity is not None:  
        result["total"] = quantity * default_price  
    return result
```

```
from typing import Optional
```

```
def get_recipe(name: str, quantity: Optional[int] = None):  
    recipe = recipes[name]  
    title = name.replace("-", " ").title()  
    result: dict = {  
        "message": f"Recipe for {title}",  
        "ingredients": recipe["ingredients"],  
    }  
    if quantity is not None:  
        result["total"] = quantity * default_price  
    return result
```





Type annotations (type hints)



With type hints



Without type hints

imgflip.com





Type annotations (type hints)

Some of the people behind:

Jukka Lehtosalo



@JukkaL



@JukkaLeh

Ivan Levkivskyi

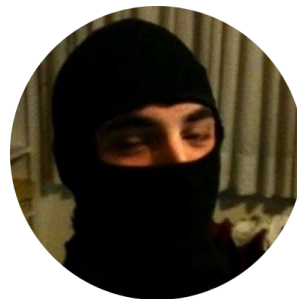


@ilevkivskyi



@ILevkivskyi

Michael J. Sullivan



@msullivan



@msully4321



@tiangolo

Modern Python



Type annotations with

 FastAPI





Type annotations with **FastAPI**

```
from typing import Optional
from fastapi import FastAPI
```

```
recipes = {
    "crunchy-frog": {
        "ingredients": [
            "frogs",
            "dew",
            "spring water",
            "cream milk chocolate",
            "glucose",
        ]
    },
    "albatross": {
        "ingredients": ["albatross"]
    }
}
default_price = 1
```

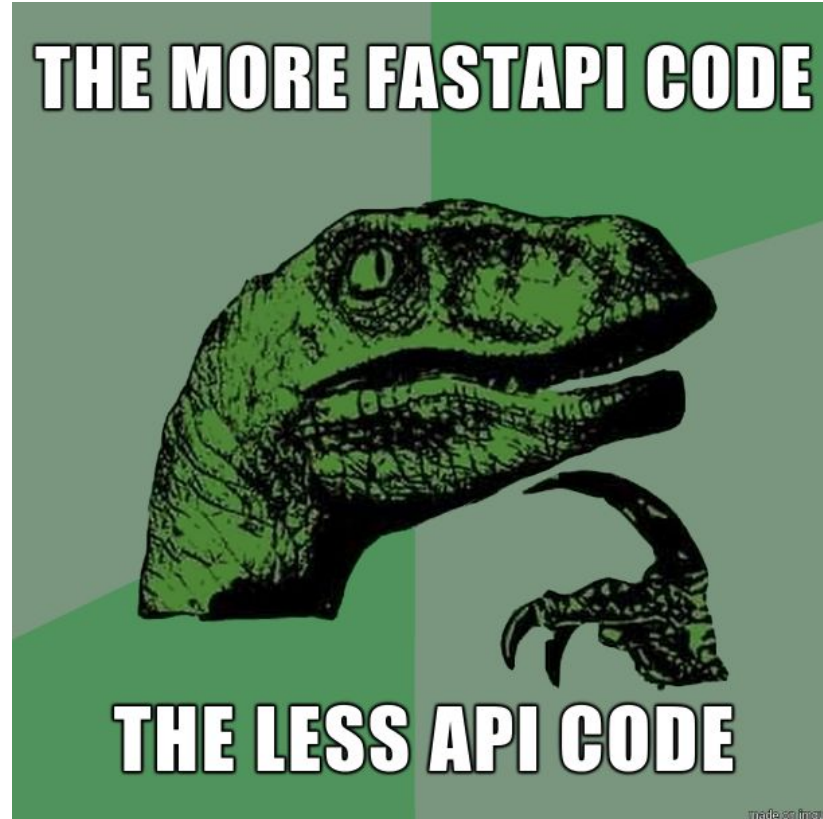
```
app = FastAPI()
```

```
@app.get("/recipes/{name}")
def get_recipe(name: str, quantity: Optional[int] = None):
    recipe = recipes[name]
    title = name.replace("-", " ").title()
    result: dict = {
        "message": f"Recipe for {title}",
        "ingredients": recipe["ingredients"],
    }
    if quantity is not None:
        result["total"] = quantity * default_price
    return result
```





Type annotations with **FastAPI**





Type annotations with **FastAPI** - autocompletion

```
@app.get("/recipes/{name}")
def get_recipe(name: str, quantity: Optional[int] = None):
    recipe = recipes[name]
    title = name.replace("-", " ").
    result: dict = {
        "message": f"Recipe for {ti
        "ingredients": recipe["ingr
    }
    if quantity is not None:
        result["total"] = quantity
    return result
```

- title
- capitalize
- casefold
- center
- count
- encode
- endswith
- expandtabs
- find





Type annotations with **FastAPI** - autocompletion





Type annotations with **FastAPI** - error detection

```
@app.get("/recipes/{name}")
def get_recipe(name: str, quantity: Optional[int] = None):
    recipe = recipes[name]
    title = (parameter) quantity: int | None
    result:
        "mes
        "ing
    }
    total = quantity * default_price
    if quantity is not None:
        result["total"] = total
    return result
```

Unsupported operand types for * ("None" and "int") mypy(error)

Left operand is of type "Optional[int]" mypy(note)

Peek Problem (Alt+F8) No quick fixes available



FastAPI - API documentation

FastAPI 0.1.0 OAS3
[/openapi.json](#)

<http://127.0.0.1:8000/docs>

default ▼

GET

/recipes/{name}

Get Recipe

Parameters

Cancel

Name	Description
name ★ required string (path)	crunchy-frog
quantity integer (query)	quantity

Execute

Clear



FastAPI - API documentation

Responses

Curl

```
curl -X GET "http://127.0.0.1:8000/recipes/crunchy-frog" -H "accept: application/json"
```

Request URL

```
http://127.0.0.1:8000/recipes/crunchy-frog
```

Server response

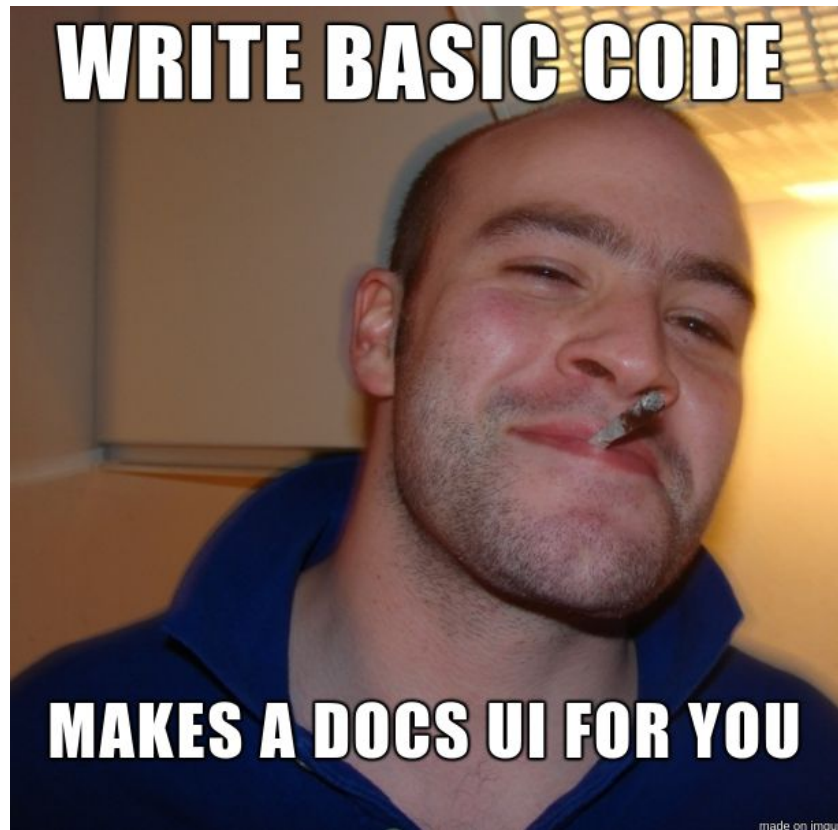
Code	Details
200	Response body <pre>{ "message": "Recipe for Crunchy Frog", "ingredients": ["frogs", "dew", "spring water", "cream milk chocolate", "glucose"]}</pre> Response headers <pre>content-length: 115content-type: application/jsondate: Sat, 21 Nov 2020 12:59:43 GMTserver: uvicorn</pre>

Responses

Code	Description	Links
200	Successful Response	No links



FastAPI - API documentation



FastAPI - data validation

FastAPI 0.1.0 OAS3

[/openapi.json](#)

default



GET `/recipes/{name}` Get Recipe

Parameters Cancel

Name	Description
name ★ required string (path)	crunchy-frog
quantity integer (query)	nine

Execute Clear



@tiangolo

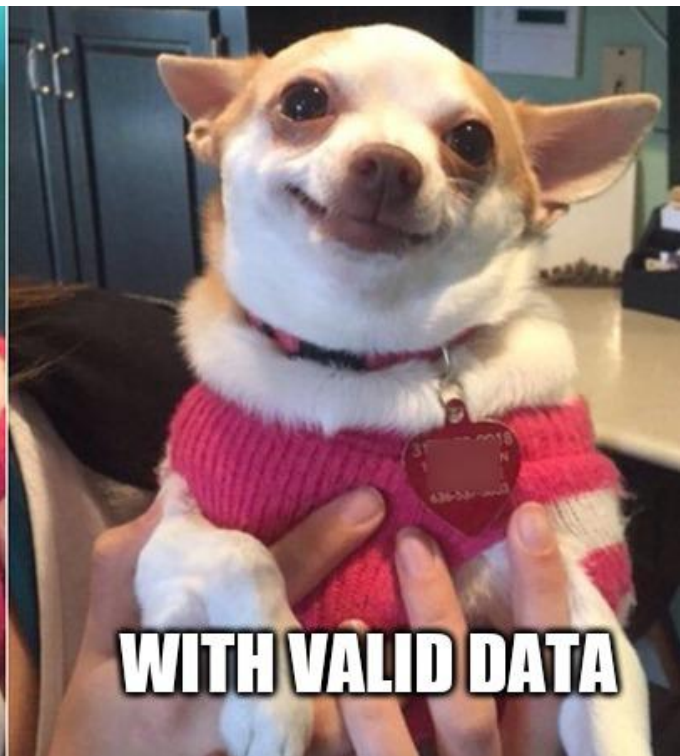
FastAPI - data validation

```
> curl "http://127.0.0.1:8000/recipes/crunchy-frog?quantity=nine"  
{  
  "detail": [  
    {  
      "loc": [  
        "query",  
        "quantity"  
      ],  
      "msg": "value is not a valid integer",  
      "type": "type_error.integer"  
    }  
  ]  
}
```

```
{  
  "detail": [  
    {  
      "loc": [  
        "query",  
        "quantity"  
      ],  
      "msg": "value is not a valid integer",  
      "type": "type_error.integer"  
    }  
  ]  
}
```



FastAPI - data validation



FastAPI - data conversion

FastAPI 0.1.0 OAS3

/openapi.json

default



GET

/recipes/{name} Get Recipe

Parameters

Cancel

Name	Description
name ★ required	crunchy-frog
quantity	2

string
(path)

integer
(query)

http://127.0.0.1:8000/recipes/crunchy-frog?quantity=2

Execute

Clear



@tiangolo

FastAPI - data conversion

Code	Details
200	Response body <pre>{ "message": "Recipe for Crunchy Frog", "ingredients": ["frogs", "dew", "spring water", "cream milk chocolate", "glucose"], "total": 2 }</pre>  Download



FastAPI - Type hints



FastAPI is based on standards

- OpenAPI
- JSON Schema
- OAuth2



Automatic API docs ✨

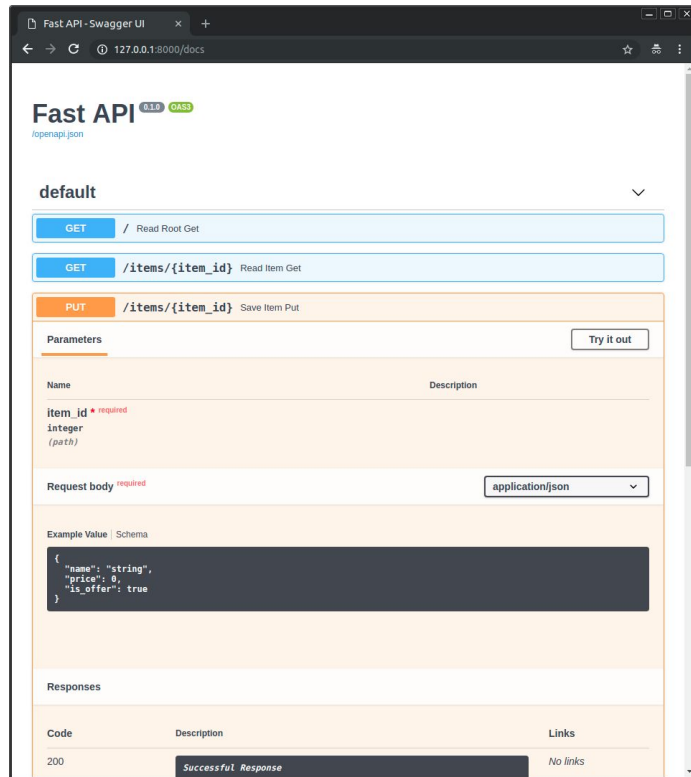
- Standard type annotations



Automatic data conversion ✨



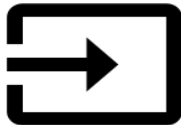
Automatic data validation ✨



Modern Python



Type annotations with

 **Typing**



Type annotations with **Typer**

```
from typing import Optional
import typer
```

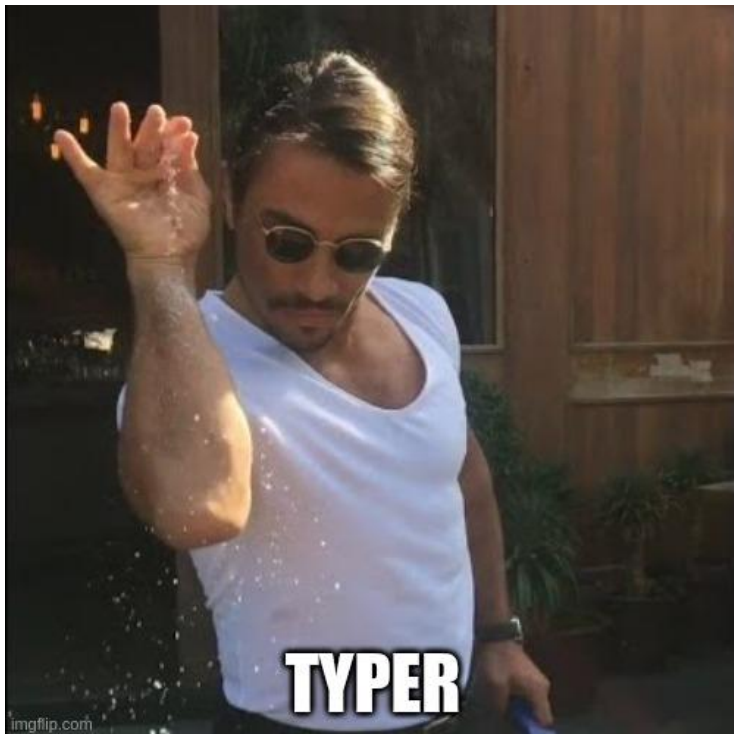
```
recipes = {
    "crunchy-frog": {
        "ingredients": [
            "frogs",
            "dew",
            "spring water",
            "cream milk chocolate",
            "glucose",
        ]
    },
    "albatross": {
        "ingredients": ["albatross"]},
}
default price = 1
```

```
app = typer.Typer()
```

```
@app.command()
def get_recipe(name: str, quantity: Optional[int] = None):
    recipe = recipes[name]
    title = name.replace("-", " ").title()
    result: dict = {
        "message": f"Recipe for {title}",
        "ingredients": recipe["ingredients"],
    }
    if quantity is not None:
        result["total"] = quantity * default_price
    typer.echo(result)
```



Type annotations with **Typer**



Typer - automatic help errors

```
> python main.py
Usage: main.py [OPTIONS] NAME
Try 'main.py --help' for help.

Error
Missing argument 'NAME'.
```



Typer - CLI Arguments

```
> python main.py crunchy-frog  
{'message': 'Recipe for Crunchy Frog', 'ingredients': ['frogs',  
'dew', 'spring water', 'cream milk chocolate', 'glucose']}
```



Typer - CLI Options

```
> python main.py crunchy-frog --quantity 2  
{'message': 'Recipe for Crunchy Frog', 'ingredients': ['frogs',  
'dew', 'spring water', 'cream milk chocolate', 'glucose'],  
'total': 2}
```



Typing - validation

```
> python main.py crunchy-frog --quantity nine
```

```
Usage: main.py [OPTIONS] NAME
```

```
Try 'main.py --help' for help.
```

```
Error
```

```
Invalid value for '--quantity': 'nine' is not a valid integer.
```



Typer - shell (TAB) completion

```
> typer ./main.py run --q
```

<TAB>

```
> typer ./main.py run --quantity
```



Typer



Modern Python



Type annotations with

 **Typ**er

and Rich



Typer with Rich

```
from typing import Optional
from typer import Typer
from rich.table import Table
from rich.console import Console
```

```
recipes = {
    "crunchy-frog": {
        "ingredients": [
            "frogs",
            "dew",
            "spring water",
            "cream milk chocolate",
            "glucose",
        ]
    },
    "albatross": {
        "ingredients": ["albatross"]},
}
default_price = 1
```

```
app = Typer()
console = Console()
```

```
@app.command()
def get_recipe(name: str, quantity: Optional[int] = None):
    recipe = recipes[name]
    title = name.replace("-", " ").title()
    table = Table("Ingredients", title=title)
    for ingredient in recipe["ingredients"]:
        table.add_row(ingredient)
    console.print(table)
    if quantity is not None:
        total = quantity * default_price
        console.print(f"Total: {total}", style="bold")
```



Typer with Rich

```
> python main.py crunchy-frog --quantity 2
```

Crunchy Frog
Ingredients
frogs dew spring water cream milk chocolate glucose

Total: 2



Typer with Rich



Typer and friends - the people behind

David Lord
Click maintainer



Will McGugan
Rich creator



Modern Python



Type annotations with

 **FastAPI**

and Pydantic



FastAPI with Pydantic

```
from typing import List
from fastapi import FastAPI
from pydantic import BaseModel
```

```
class Food(BaseModel):
    name: str
    ingredients: List[str] = []
```

```
app = FastAPI()
```

```
@app.post("/food/")
def prepare_food(food: Food):
    return {
        "message": f"New food added: {food.name}",
        "total_ingredients": len(food.ingredients)
    }
```



FastAPI with Pydantic

```
from typing import List
from fastapi import FastAPI
from pydantic import BaseModel
```

```
class Food(BaseModel):
    name: str
    ingredients: List[str] = []
```

```
app = FastAPI()
```

```
@app.post("/food/")
def prepare_food(food: Food):
    return {
        "message": f"New food added: {food.name}",
        "total_ingredients": len(food.ingredients)
    }
```



FastAPI with Pydantic - API documentation

FastAPI 0.1.0 OAS3

[/openapi.json](#)

default



POST

/food/ Prepare Food

Parameters

Try it out

No parameters

Request body required

application/json



Example Value | Schema

```
{
  "name": "string",
  "ingredients": []
}
```



@tiangolo

FastAPI with Pydantic - interactive API docs

POST **/food/** Prepare Food

Parameters

No parameters

Request body required

application/json

```
{
  "name": "albatross",
  "ingredients": ["albatross"]
}
```

Execute

Clear



FastAPI with Pydantic - interactive API docs

Code	Details
200	Response body <pre>{ "message": "New food added: albatross", "total_ingredients": 1 }</pre>  Download Response headers <pre>content-length: 61 content-type: application/json date: Sat, 21 Nov 2020 17:29:03 GMT server: uvicorn</pre>



FastAPI with Pydantic - data validation

POST **/food/** Prepare Food

Parameters Cancel

No parameters

Request body required application/json

```
{
  "name": "albatross",
  "ingredients": [
    "albatross",
    {"name": "frogs"}
  ]
}
```

Execute Clear



FastAPI with Pydantic - data validation

Code	Details
422	Error: Unprocessable Entity Response body <pre>{ "detail": [{ "loc": ["body", "ingredients", 1], "msg": "str type expected", "type": "type_error.str" }] }</pre>  Download



FastAPI with Pydantic - nested data

```
from typing import List
from fastapi import FastAPI
from pydantic import BaseModel
```

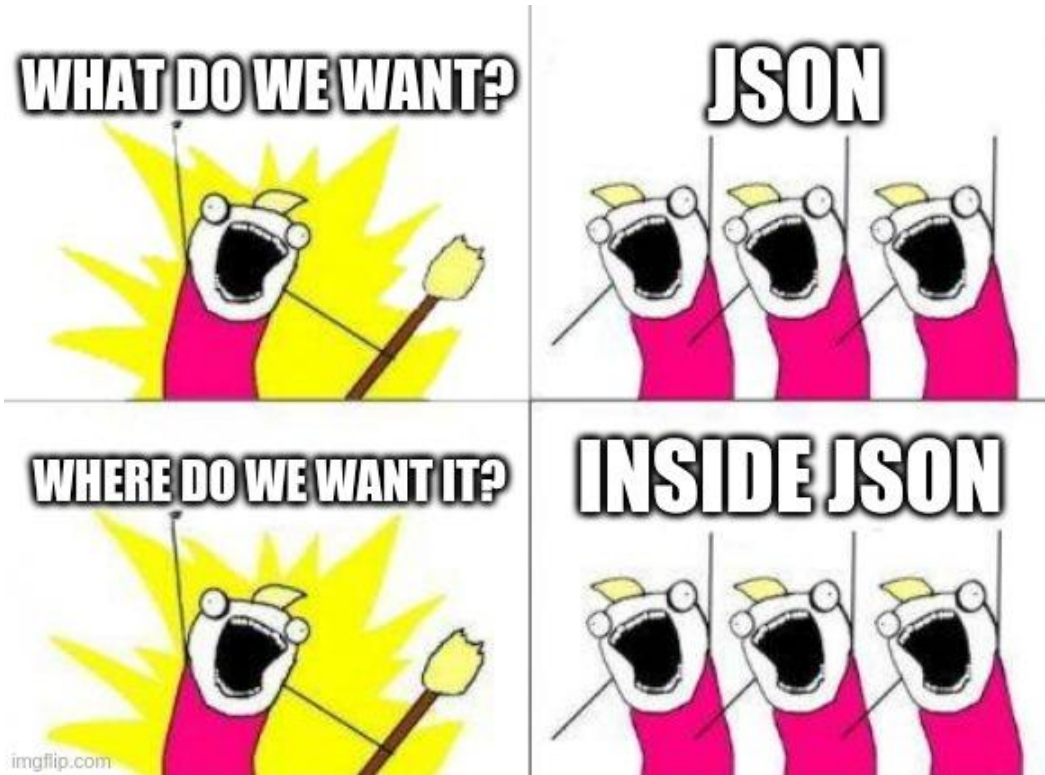
```
app = FastAPI()
```

```
class Food(BaseModel):
    name: str
    ingredients: List[str] = []
```

```
@app.post("/food/")
def prepare_food(orders: List[Food]):
    all_ingredients = []
    for food in orders:
        for ingredient in food.ingredients:
            all_ingredients.append(ingredient.lower())
    return {"ingredients": all_ingredients}
```



FastAPI with Pydantic - nested data



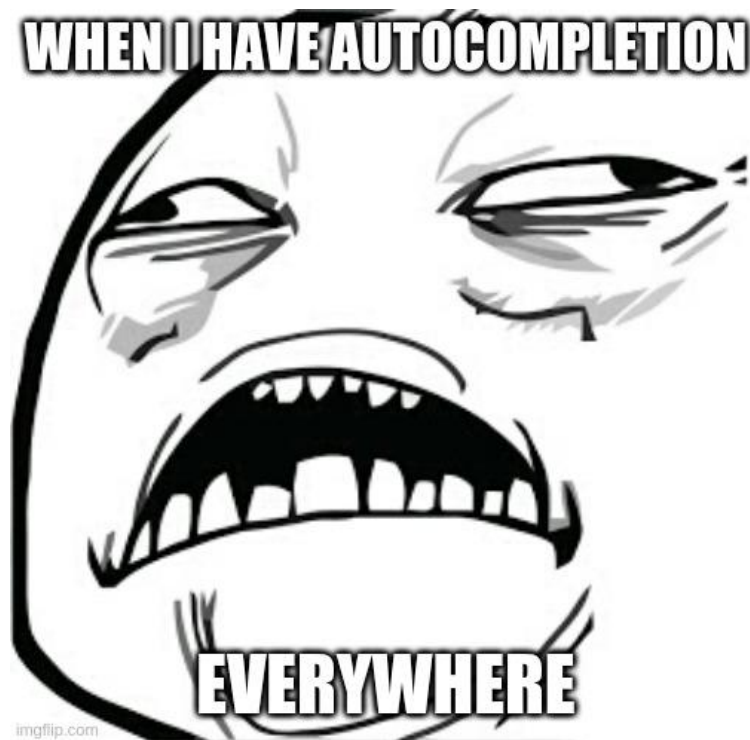
FastAPI with Pydantic - autocompletion

```
@app.post("/food/")
def prepare_food(orders: List[Food]):
    all_ingredients = []
    for food in orders:
        for ingredient in food.ingredients:
            all_ingredients.append(ingredient.)
    return {"ingredients": all_ingredients}
```

- lower
- ljust
- title
- capitalize
- casefold
- center
- count



FastAPI with Pydantic - autocompletion



FastAPI with Pydantic - error detection

```
@app.post("/food/")
def prepare_food(orders: List[Food]):
    all_ingredients = []
    for food in orders:
        for ingredient in ingredient + 9000
        all_ingredients.append(ingredient.lower())
    return {"ingredients": all_ingredients}
```

Unsupported operand types for + ("str" and "int") mypy(error)

Peek Problem (Alt+F8) No quick fixes available



FastAPI - the people behind

Samuel Colvin

Pydantic creator



@samuelcolvin



@samuel_colvin

Tom Christie

Creator of Starlette (and more)



@tomchristie



@starletdreaming

David Montague

Pydantic and FastAPI
notorious contributor



@dmontagu



@tiangolo

Modern Python



Type annotations with

 **FastAPI**

and

 **SQLModel**



FastAPI with SQLAlchemy

```
from typing import List, Optional

from fastapi import FastAPI
from sqlalchemy import Field, Session, SQLAlchemy, create_engine, select

class Hero(SQLAlchemy, table=True):
    id: Optional[int] = Field(default=None, primary_key=True)
    name: str
    secret_name: str
    age: Optional[int] = None

engine = create_engine(
    "sqlite:///database.db", connect_args={"check_same_thread": False}
)
```



FastAPI with SQLAlchemy

```
app = FastAPI()

@app.on_event("startup")
def on_startup():
    SQLAlchemy.metadata.create_all(engine)

@app.post("/heroes/", response_model=Hero)
def create_hero(hero: Hero):
    with Session(engine) as session:
        session.add(hero)
        session.commit()
        session.refresh(hero)
    return hero
```



FastAPI with SQLAlchemy

```
@app.get("/heroes/", response_model=List[Hero])
def read_heroes():
    with Session(engine) as session:
        heroes = session.exec(select(Hero)).all()
        return heroes
```



FastAPI with SQLAlchemy

FastAPI 0.1.0 OAS3

/openapi.json

default

GET

/heroes/

Read Heroes

POST

/heroes/

Create Hero

Parameters

Try it out

No parameters

Request body required

application/json

Example Value | Schema

```
{
  "id": 0,
  "name": "string",
  "secret_name": "string",
  "age": 0
}
```



@tiangolo

FastAPI with SQLAlchemy



SQLModel - the people behind

Samuel Colvin

Pydantic creator



@samuelcolvin



@samuel_colvin

Mike Bayer

SQLAlchemy creator



@zzzeek



@zzzeek



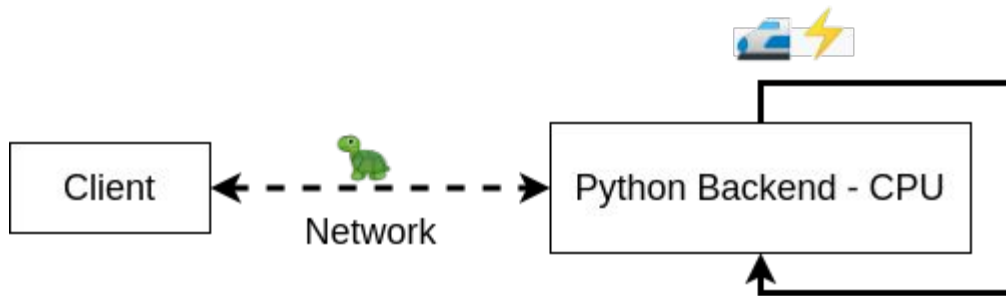
@tiangolo

Modern Python

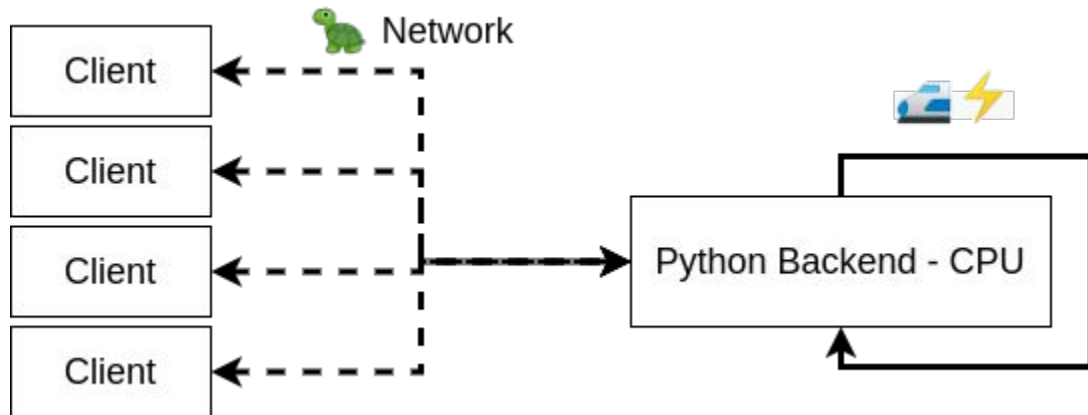
 `async / await`



async / await - concurrency



async / await - concurrency



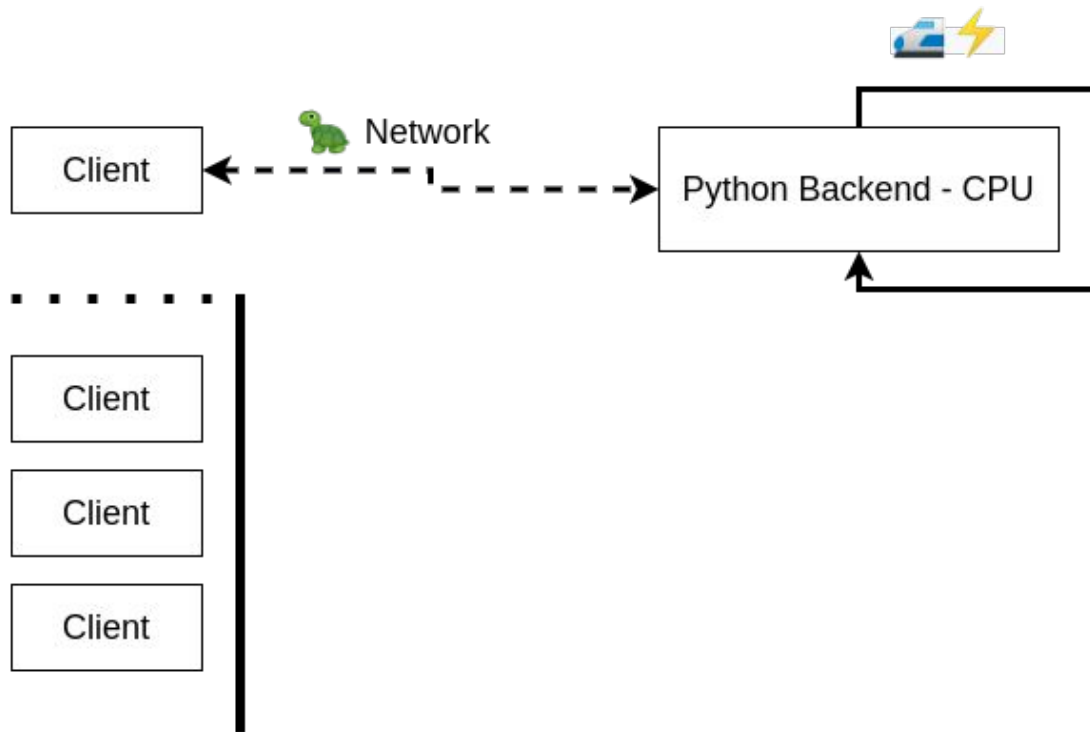
async / await - concurrency



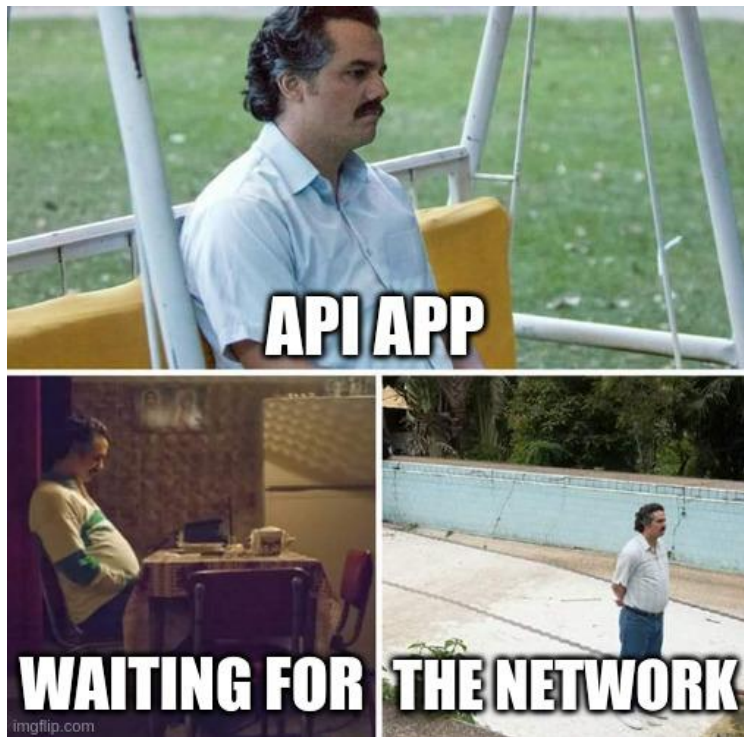
Photo by John Cameron: https://unsplash.com/@john_cameron



async / await - concurrency



🕒 async / await - concurrency



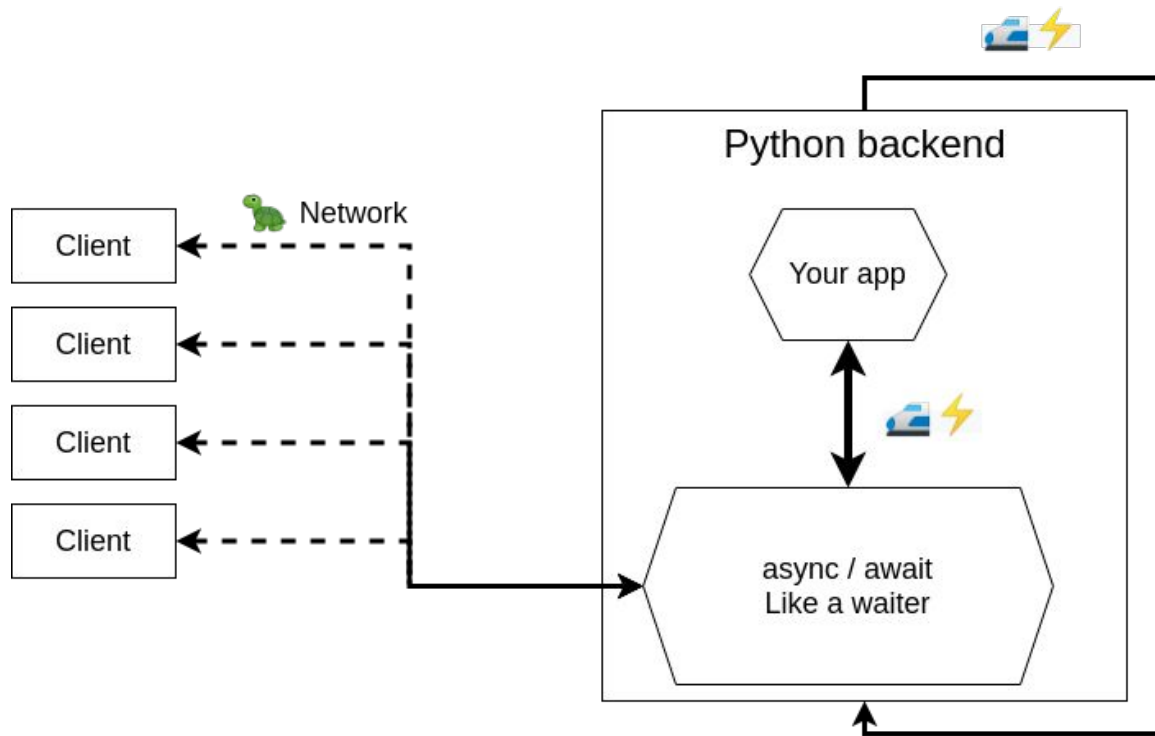
async / await - concurrency



Photo by Kate Townsend: <https://unsplash.com/@k8townsend>



async / await - concurrency





async / await - FastAPI and HTTPX

```
from fastapi import FastAPI
import httpx
```

```
app = FastAPI()
```

```
@app.get("/recipes/{name}")
async def get_recipe(name: str):
    async with httpx.AsyncClient() as client:
        response = await client.get(f"https://recipes-api.com/recipes/{name}")
        return response.json()
```





async / await - FastAPI and HTTPX

FastAPI 0.1.0 OAS3

[/openapi.json](#)

default



GET /recipes/{name} Get Recipe

Parameters

Cancel

Name	Description
name ★ required string (path)	crunchy-frog

Execute

Clear





async / await - FastAPI and HTTPX

Responses

Curl

```
curl -X GET "http://127.0.0.1:8001/recipes/crunchy-frog" -H "accept: application/json"
```

Request URL

```
http://127.0.0.1:8001/recipes/crunchy-frog
```

Server response

Code	Details
200	Response body <pre>{ "message": "Recipe for Crunchy Frog", "ingredients": ["frogs", "dew", "spring water", "cream milk chocolate", "glucose"] }</pre> Download Response headers <pre>content-length: 115 content-type: application/json date: Sat, 21 Nov 2020 19:11:58 GMT server: uvicorn</pre>





async / await - optional with **FastAPI** and **HTTPX**

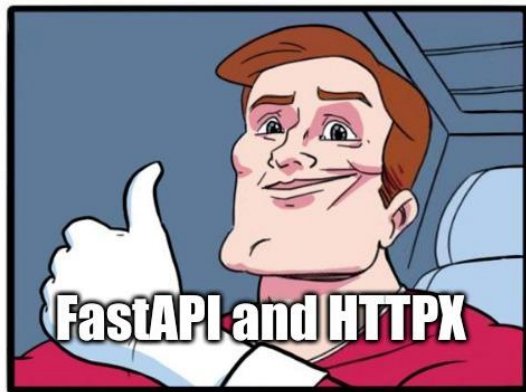
```
from fastapi import FastAPI  
import httpx
```

```
app = FastAPI()
```

```
@app.get("/recipes/{name}")  
def get_recipe(name: str):  
    response = httpx.get(f"https://recipes-api.com/recipes/{name}")  
    return response.json()
```



⌚ async / await - optional with **FastAPI** and **HTTPX**



FastAPI async / await - the people behind

Tom Christie

Creator of Starlette, HTTPX,
and more



@tomchristie



@starletdreaming

Florimond Manca

Co-maintainer of HTTPX,
and others



@florimondmanca



@florimondmanca

Andrew Godwin

Author of the ASGI
specification



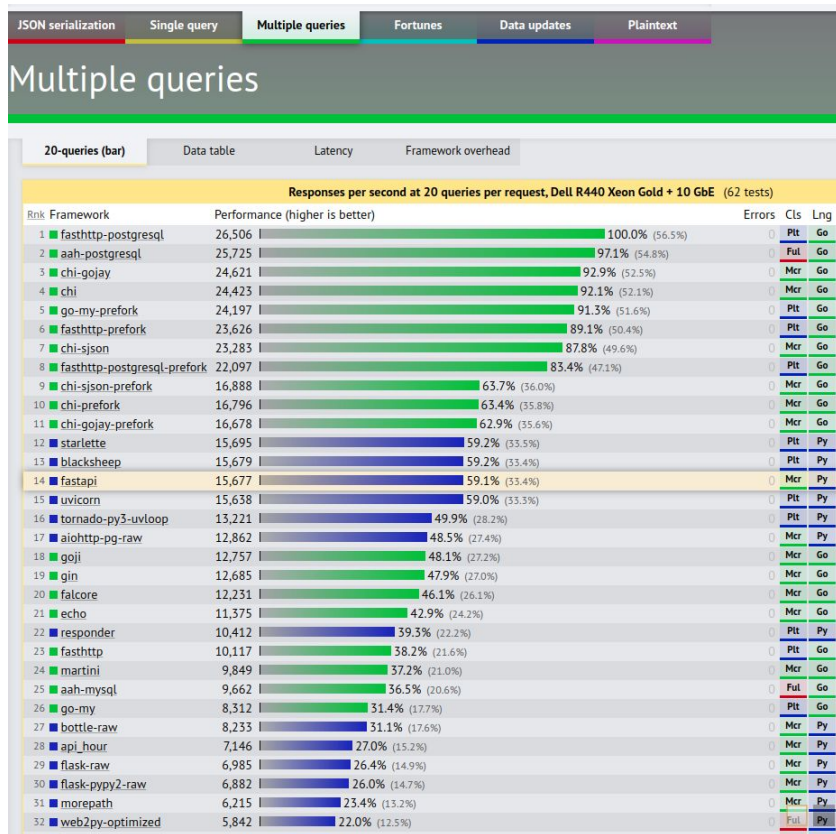
@andrewgodwin



@andrewgodwin



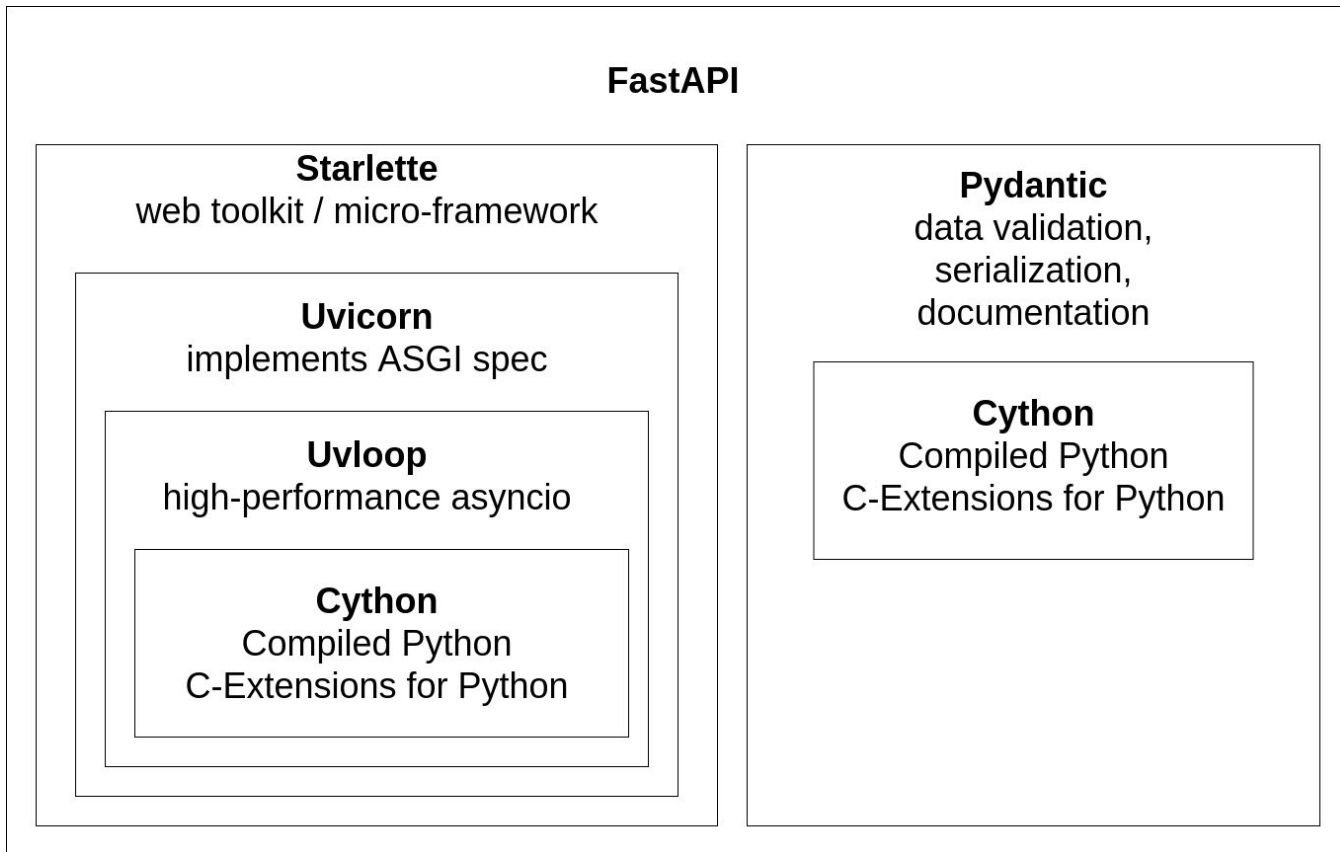
⚡ Performance with FastAPI



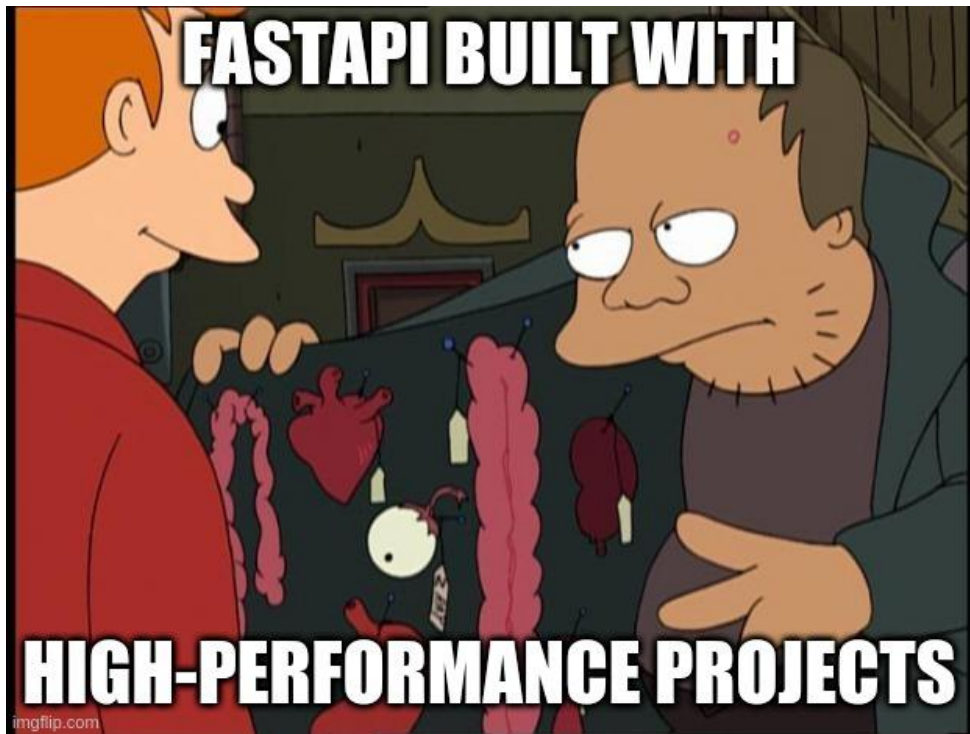
⚡ Performance with FastAPI



Performance with FastAPI



⚡ Performance with FastAPI



⚡ Performance - the people behind

Tom Christie

Creator of Starlette, Uvicorn,
and more



@tomchristie



@_tomchristie

Yury Selivanov

Creator of Uvloop



@1st1



@1st1

Stefan Behnel

Maintainer of Cython



@scoder



@tiangolo



Community

- All by normal people 
- Mostly during free time 
- Trying to help others 



Community



Community - FastAPI

You can help too! 

- Translate a page 
- Review translations from others 
- Answer questions from others 
- Help with bugs and features 
- Help with other projects 

<https://fastapi.tiangolo.com/help-fastapi/>

<https://fastapi.tiangolo.com/contributing/#translations>





Thank you!



Sebastián Ramírez

tiangolo.com

fastapi.tiangolo.com



github.com/tiangolo



linkedin.com/in/tiangolo



twitter.com/tiangolo

